

Version 1.3

January 2026



STRUKTUR DATA

MODUL 1 - GENERICS JAVA

Penulis:

MUHAMMAD ILHAM PERDANA. S.Tr.T., M.T.

MUHAMMAD IBRAHIM AL AYUBI

MARENO ALI IBRAHIM

INFORMATICS LABORATORY

University of Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	3
TUJUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KEYWORDS.....	3
REFERENSI.....	3
MATERI.....	4
1. Generics.....	4
1.1 Apa itu Generics.....	4
1.2 Mengapa Mempelajari Generics?.....	6
1.3 Perbedaan Kode dengan dan Tanpa Generics.....	6
1.4 Deklarasi Class dengan Generic Method.....	8
1.5 Konvensi Penamaan Parameter untuk Generics.....	9
1.6 Generics Method.....	9
2. Wildcards.....	10
3. Enumerated Types.....	11
CODELAB.....	13
DEMO.....	15
Penilaian.....	16
Rubric Penilaian.....	16
Grading Scale.....	16



PENDAHULUAN

TUJUAN

1. Mahasiswa mampu memahami dan menjelaskan konsep dasar Java Generics serta mengidentifikasi peran generics dalam berbagai skenario dunia nyata.

TARGET MODUL

1. Mahasiswa mampu memahami: implementasi array objek, kelas generik, metode generik, wildcard, dan tipe enumerasi.

PERSIAPAN

1. Bagi mahasiswa Muslim, dapat membaca doa berikut sebelum memulai pembelajaran.

رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Laptop atau komputer
3. Java Development Kit
4. Java Runtime Environment
5. IDE (IntelliJ IDEA, Eclipse, Netbeans, etc.)
6. Motivasi dan komitmen yang kuat untuk belajar

KEYWORDS

Generics, Type Parameter, Generic Class, Generic Method, Wildcard, Type Safety, Parameterized Type.

REFERENSI

Oracle iLearning – Java Programming, Section 6-1: Generics

GeeksforGeeks: <https://www.geeksforgeeks.org/generics-in-java/>

YouTube (Indonesian): <https://www.youtube.com/watch?v=bvWRDAI30Gs>

YouTube (English): <https://www.youtube.com/watch?v=K1iu1kXkVoA>

Javatpoint: <https://www.javatpoint.com/generics-in-java>

Programiz: <https://www.programiz.com/java-programming/generics>

Note: Referensi-referensi tersebut mungkin memiliki sedikit perbedaan dalam penjelasan atau contohnya. Fokuslah pada pemahaman konsep dasar dan penerapannya pada studi kasus yang diberikan dalam modul ini.



MATERI

Selamat datang di dunia struktur data. Sebelum kita membangun struktur data yang lebih kompleks seperti Stack atau Queue, kita harus terlebih dahulu memahami satu hal yang mendasar dalam pemrograman.

Bagaimana kita dapat menulis kode yang dapat menangani berbagai jenis data tanpa harus menulis ulang logika yang sama berulang kali?

Bayangkan kamu adalah seorang arsitek di sebuah stasiun kereta api. Kamu diminta untuk merancang sistem penyimpanan loker.

- Jika kamu merancang loker yang hanya dapat menampung tas ransel, koper berukuran besar tidak akan muat.
- Jika kamu merancang loker yang secara khusus ditujukan untuk koper, dompet berukuran kecil akan menjadi kurang praktis untuk disimpan.

Dalam pemrograman tradisional, kita sering terjebak dalam pembuatan container yang kaku seperti NumberBox, StringBox, atau ObjectBox. Hal ini tidak efisien dan menyebabkan banyak pengulangan kode. **Java Generics** hadir untuk mengatasi permasalahan tersebut. Generics dapat diibaratkan sebagai **label kosong** pada sebuah wadah. Ketika kamu merancang wadah tersebut, kamu tidak menentukan terlebih dahulu apa yang akan dimasukkan ke dalamnya. Tipe data ditentukan kemudian, saat wadah tersebut digunakan.

1. Generics

1.1 Apa itu Generics

Generics merupakan fitur Java yang memungkinkan kita menggunakan tipe data sebagai parameter saat mendefinisikan sebuah Class, Generic Method, atau Method. Secara sederhana, Generics memungkinkan kita membuat sebuah Class yang fleksibel. Tipe data yang digunakan di dalam Class tidak ditentukan secara tetap saat proses penulisan kode, tetapi dapat ditentukan kemudian saat Class tersebut dibuat (instantiated).



Contoh Kode



```

1 package com.test;
2
3 class Box<T> {
4     T value;
5
6     Box(T value) {
7         this.value = value;
8     }
9
10    void show() {
11        System.out.println(value);
12    }
13 }
14
15 public class Main {
16     public static void main(String[] args) {
17         Box<String> b = new Box<>("Ticket");
18         b.show();
19     }
20 }
21

```

Output

```

~/Downloads/drive-download-20251202T110254Z-1-001/Modul/1
• > cd /home/mipow/Downloads/drive-download-20251202T110254Z-1-001/Modul/1
tionMessages -cp /home/mipow/Downloads/drive-download-20251202T110254Z-1-001/Modul/1
Ticket

```



1.2 Mengapa Mempelajari Generics?

Mempelajari Generics sangat penting dalam pengembangan perangkat lunak modern karena dua alasan utama: keamanan dan efisiensi.

Pertama, Generics memberikan Type Safety. Tanpa Generics, kesalahan tipe data sering kali baru terdeteksi saat program dijalankan (runtime), yang dapat menyebabkan aplikasi crash pada pengguna. Dengan Generics, kesalahan tersebut dapat dideteksi oleh compiler saat kita menulis kode (Compile-Time Error), sehingga jauh lebih aman.

Kedua, Generics memungkinkan Code Reusability. Sebagai seorang programmer, kamu tidak perlu membuang waktu membuat Class yang berbeda seperti `WagonString`, `WagonInteger`, atau `WagonDouble` yang sebenarnya memiliki logika yang sama. Kamu cukup membuat satu Class `Wagon<T>` yang dapat digunakan untuk berbagai tipe data. Konsep ini menjadi dasar dari struktur data modern seperti Stack, Queue, dan Linked List yang akan kita pelajari selanjutnya.

1.3 Perbedaan Kode dengan dan Tanpa Generics

Dulu (sebelum Java 5), programmer mengandalkan tipe Object sebagai container universal. Pendekatan ini seperti sebuah kotak misterius yang dapat digunakan untuk menyimpan String, Integer, maupun objek lainnya tanpa membedakan jenisnya. Meskipun fleksibel, pendekatan ini memiliki risiko Type Safety yang cukup besar. Ketika mengambil data dari container tersebut, compiler tidak mengetahui tipe objek yang sebenarnya. Akibatnya, kita harus melakukan casting atau konversi tipe data secara manual. Jika kita salah mengingat tipe data yang disimpan, misalnya mencoba memperlakukan String yang tersimpan sebagai Integer, program dapat crash saat dijalankan dan menghasilkan `ClassCastException`.

TANPA GENERICS (Tipe: Object)

MEMORI HEAP:



PROSES PENGAMBILAN DATA:



Mengambil data sebagai **Object**.



Apa isi di dalamnya?
(Tidak diketahui)



Harus melakukan Casting
(Berisiko menyebabkan Crash).

DENGAN GENERICS (Tipe: <T>)

MEMORI HEAP:



PROSES PENGAMBILAN DATA:



Mengambil data secara langsung sebagai **String**.



Tidak memerlukan Casting.



Siap digunakan secara langsung.



Di bawah ini adalah contoh kode sebelum menggunakan Generics (cara lama). Perhatikan bahwa kita harus menebak tipe data dan melakukan casting secara manual, sehingga sangat rentan terhadap runtime error yang dapat menyebabkan program crash.

```
// "SEBELUM" CODE: Tanpa Generics (Menggunakan Object)
class OldWagon {
    private Object cargo; // Tipe Object dapat menyimpan apa saja

    public void setCargo(Object cargo) {
        this.cargo = cargo;
    }

    public Object getCargo() {
        return cargo;
    }
}
```

```
// Simulasi pada Main Class:
public class Main {
    public static void main(String[] args) {

        OldWagon wagon = new OldWagon();

        // 1. Kita memasukkan String ("Cow")
        wagon.setCargo("Cow");

        // 2. Saat mengambil data, kita HARUS melakukan casting secara manual
        // Masalah fatal: Jika programmer lupa dan menganggap ini
        // Number...
        Integer number = (Integer) wagon.getCargo();

        // HASIL: Program mengalami CRASH saat runtime
        // (ClassCastException)
    }
}
```

Kemudian bandingkan dengan pendekatan modern menggunakan Generics yang menerapkan Type Safety secara ketat. Dengan menentukan tipe data di awal menggunakan <...>, compiler dapat bertindak sebagai penjaga yang ketat. Hanya data dengan tipe yang sesuai yang dapat dimasukkan ke dalam container. Di bawah ini adalah contoh kode setelah menggunakan Generics. Perhatikan bahwa kita tidak perlu lagi melakukan casting dan kesalahan tipe data dapat dicegah oleh compiler sebelum program dijalankan.

```
// "SETELAH" CODE: Dengan Generics (Type Safety)
```



```
class ModernWagon<T> {
    private T cargo; // Tipe data akan menyesuaikan dengan tipe order

    public void setCargo(T cargo) {
        this.cargo = cargo;
    }

    public T getCargo() {
        return cargo;
    }
}
```

```
// Simulasi pada Main Class:
public class Main {
    public static void main(String[] args) {

        // Kita memastikan wagon ini hanya untuk String
        ModernWagon<String> wagon = new ModernWagon<>();

        // 1. Memasukkan data yang valid (Aman)
        wagon.setCargo("Cow");

        // 2. Tidak memerlukan casting secara manual.
        // Data secara otomatis dikenali sebagai String
        String content = wagon.getCargo();
        System.out.println("Content: " + content);

        // 3. Pencegahan Error
        // Jika kita mencoba memasukkan Integer, Compiler akan menolaknya
        // wagon.setCargo(100); // <-- COMPILE ERROR! (Aman)
    }
}
```

1.4 Deklarasi Class dengan Generic Method

Mendeklarasikan Generic Class melibatkan perubahan sintaks sederhana tetapi sangat kuat: tambahkan tanda kurung sudut yang berisi type parameter (umumnya <T>) tepat setelah nama Class. Huruf T berfungsi sebagai placeholder atau variabel tipe untuk tipe data yang akan ditentukan kemudian saat objek dibuat. Hal ini bertindak seperti sebuah kontrak: setiap kali T muncul sebagai atribut, parameter method, atau tipe nilai yang dikembalikan, T akan digantikan dengan tipe data sebenarnya seperti String, Integer, atau Class khusus seperti Passenger.



```
// Contoh 1: 1 Type Parameter
public class Wagon<T> { ..... }

// Contoh 2: 2 Type Parameter (misalnya, Ticket dengan Code & Passenger)
public class Ticket<K, V> {
    private K code;
    private V passenger;

    public Ticket(K code, V passenger) {
        this.code = code;
        this.passenger = passenger;
    }
}

return go(f, seed, [])
}
```

Fleksibilitas ini tidak terbatas pada satu parameter saja. Untuk struktur data yang lebih kompleks, seperti map yang membutuhkan pasangan key-value, kamu dapat mendeklarasikan beberapa variabel tipe yang dipisahkan dengan koma, misalnya <K, V>. Dalam konteks ini, K biasanya merepresentasikan tipe Key, sedangkan V merepresentasikan tipe Value. Ketika Class digunakan, programmer memberikan tipe data yang sesuai untuk keduanya, sehingga memungkinkan terbentuknya hubungan data yang terstruktur dan memiliki Type Safety.

1.5 Konvensi Penamaan Parameter untuk Generics

Meskipun Java mengizinkan nama apa pun digunakan sebagai tipe parameter, komunitas developer umumnya mengikuti konvensi penamaan dengan satu huruf. Praktik ini penting untuk menjaga keterbacaan kode karena memungkinkan developer dengan cepat membedakan tipe parameter (placeholder) dari nama Class biasa.

- E - Element (digunakan oleh Java Collections Framework)
- K - Key
- N - Number
- T - Type
- V - Value

1.6 Generics Method

Generics tidak terbatas pada deklarasi tingkat Class; Generics juga dapat diterapkan pada method individual untuk membuat logika yang fleksibel dan dapat digunakan kembali, terlepas dari Class tempat method tersebut berada. Sebuah Generic Method mendeklarasikan type parameter-nya sendiri, yang ditempatkan sebelum tipe return. Hal ini memungkinkan method menerima argumen dengan tipe data yang berbeda pada Class yang berbeda.



Konsep ini dapat diibaratkan sebagai alat universal yang dapat menyesuaikan diri dengan berbagai ukuran baut. Sebagai contoh, sebuah utility method yang dirancang untuk mencetak isi sebuah array dapat menangani array Integer pada satu pemanggilan dan array String pada pemanggilan lainnya, tanpa memerlukan method overloading atau pengulangan kode.

```

public class PrinterUtility {

    // Method ini dapat mencetak array dengan TIPE APA PUN
    // <E> dideklarasikan sebelum tipe return void
    public static <E> void printArray(E[] input) {
        for (E element : input) {
            System.out.print(element + " ");
        }

        System.out.println();
    }
}

```

2. Wildcards

Dalam beberapa situasi, kita membutuhkan kode yang dapat berinteraksi dengan objek Generic tanpa mengetahui tipe argument secara spesifik, atau ketika kita ingin menangani berbagai tipe Generic yang berbeda. Di sinilah Wildcard, yang ditandai dengan tanda tanya <?>, menjadi penting. Wildcard digunakan untuk merepresentasikan tipe yang tidak diketahui, sehingga sebuah method dapat menerima berbagai jenis Generic, misalnya Wagon<String> maupun Wagon<Integer>.

Namun, fleksibilitas ini memiliki batasan keamanan yang ketat yang dikenal sebagai prinsip "Get and Put". Karena compiler tidak dapat memastikan tipe sebenarnya dari objek yang menggunakan wildcard, objek tersebut diperlakukan sebagai read-only. Kamu tetap dapat mengambil dan memeriksa data dengan aman karena setiap objek di Java setidaknya merupakan turunan dari kelas Object. Namun, kamu tidak diperbolehkan menambahkan atau mengubah data pada objek tersebut. Pembatasan ini mencegah terjadinya corruption tipe tersebut. Sebagai contoh, pembatasan tersebut mencegah kita secara tidak sengaja memasukkan integer ke dalam sebuah list yang sebenarnya berisi string.



```

public class StationInspection {

    // Method ini menerima Wagon dengan tipe isi apa pun
    public static void inspectWagon(Wagon<?> w) {

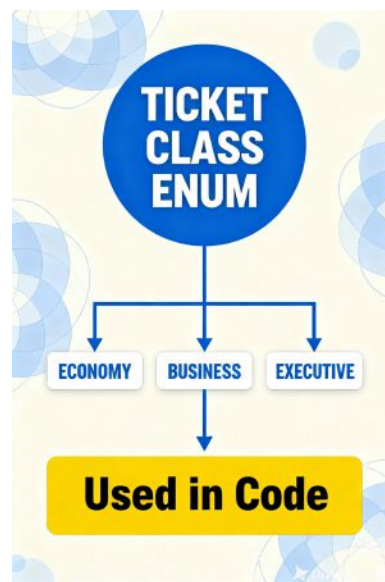
        // READ diperbolehkan
        System.out.println("Inspecting cargo: " + w.getCargo());

        // WRITE tidak diperbolehkan
        // w.setCargo("New Item"); // <-- AKAN MENYEBABKAN COMPILER ERROR
    }
}

```

3. Enumerated Types

Enumerated Types, atau Enum, menyediakan mekanisme yang kuat untuk mendefinisikan sekumpulan konstanta bernama yang nilainya tetap dan saling berkaitan. Daripada menggunakan String atau Integer secara bebas yang rentan terhadap kesalahan penulisan dan input yang tidak valid (misalnya menulis "**ekonomi**" dengan huruf kecil ketika sistem mengharapkan "**ECONOMY**"), Enum memastikan Type Safety dengan membatasi nilai yang dapat digunakan hanya pada daftar yang telah ditentukan sebelumnya.



Dalam konteks sistem perkeretaapian, atribut seperti kelas tiket atau kategori muatan merupakan contoh yang tepat untuk menggunakan Enum. Dengan mendefinisikannya sebagai ECONOMY, BUSINESS, atau EXECUTIVE, kita memastikan sistem menolak nilai yang tidak didefinisikan sejak saat Compile Time. Hal ini membuat kode lebih mudah dibaca, lebih mudah dipelihara, dan lebih andal karena compiler memastikan bahwa hanya konstanta yang telah ditentukan yang dapat digunakan di seluruh aplikasi.

```
// 1. Mendefinisikan enum
public enum FoodCategory {
    FOOD, BEVERAGE, DESSERT
}

// 2. Menggunakan enum dalam class
public class FoodOrder {

    private FoodCategory category;

    public FoodOrder(FoodCategory category) {
        this.category = category;
    }
}

// 3. Penggunaan pada Main
// Compiler memastikan kita memilih kategori yang valid
FoodOrder order = new FoodOrder(FoodCategory.FOOD);
```



CODELAB

Codelab menyediakan sebuah program Java yang berfungsi penuh untuk mengilustrasikan penerapan dasar Java Generics. Lengkapi kode berikut agar program dapat berjalan dengan benar.

```

public class GenericTicket<.....> {

    private T ticketCode;
    private String passengerName;

    public GenericTicket(..... ticketCode, String passengerName) {
        this.ticketCode = ticketCode;
        this.passengerName = passengerName;
    }

    public T getTicketCode() {
        .....
    }

    public String getPassengerName() {
        .....
    }

    public void displayTicket() {
        System.out.println("=== Ticket Information ===");
        System.out.println("Ticket Code      : " + ticketCode);
        System.out.println("Passenger Name   : " + passengerName);
        System.out.println("Ticket Code Type : " + ticketCode.getClass().getSimpleName());
    }

    public static void main(String[] args) {

        GenericTicket<String> ticket1 = new GenericTicket<>("TK-001", "Andi");

        GenericTicket<Integer> ticket2 = new GenericTicket<>(1002, "Budi");

        ticket1.displayTicket();

        System.out.println();

        ticket2.displayTicket();
    }
}

```



Lengkapi dan jalankan program implementasi Struktur Data menggunakan Java Generics di atas. Berikut adalah output yang diharapkan:

```
=== Railway Ticket Information ===  
Booking Code      : KA-001  
Passenger Name    : Andi  
Booking Code Type: String  
  
=== Railway Ticket Information ===  
Booking Code      : 1002  
Passenger Name    : Budi  
Booking Code Type: Integer
```



DEMO

Sepanjang praktikum ini, kamu akan mengembangkan Food Ordering System secara bertahap. Sistem ini akan **terus dikembangkan pada setiap modul** seiring dengan kamu mempelajari dan menerapkan konsep-konsep struktur data baru.

Pada modul ini, kamu akan membangun model data dasar untuk sistem menggunakan Java Generics.

Mahasiswa harus mengimplementasikan persyaratan berikut:

1. Buat sebuah Generic Class bernama Customer<T> untuk menyimpan informasi pelanggan, termasuk nama dan ID pelanggan.
2. Buat sebuah Class bernama FoodOrder untuk merepresentasikan pesanan makanan milik pelanggan.
3. Definisikan sebuah Enum bernama FoodCategory dengan konstanta berikut: FOOD, BEVERAGE, and DESSERT.
4. Implementasikan setidaknya satu method yang menggunakan Wildcard (<?>) untuk menampilkan informasi pelanggan atau pesanan.

Output :

<pre> === Food Ordering System === Enter Customer Name: Ayub Enter Customer ID: 12345678 Enter Order Code: ODR-01 Enter Menu Name: Ayam Geprek Select Food Category: 1. FOOD 2. BEVERAGE 3. DESSERT Enter choice: 1 === Order Information === Order Code : ODR-01 Customer Name : Ayub Customer ID : 12345678 Customer ID Type : Long Menu Name : Ayam Geprek Food Category : FOOD </pre>	<pre> === Food Ordering System === Enter Customer Name: Reno Enter Customer ID: 87654321 Enter Order Code: ODR-02 Enter Menu Name: Es Krim Select Food Category: 1. FOOD 2. BEVERAGE 3. DESSERT Enter choice: 3 === Order Information === Order Code : ODR-02 Customer Name : Reno Customer ID : 87654321 Customer ID Type : Long Menu Name : Es Krim Food Category : DESSERT </pre>
---	--

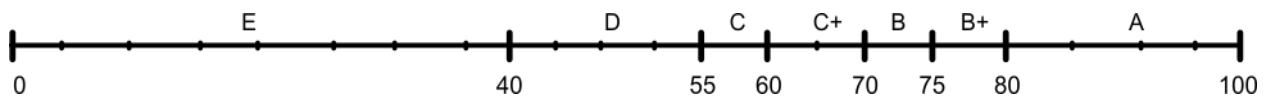


Penilaian

Rubric Penilaian

Assessment Aspect	Points (Total 100%)
Codelab	Total 20%
Successfully run	20%
Demo	Total 80%
Implement Generic	15%
Implement Wildcards	15%
Implement Enum for ticket class	15%
Running Smoothly	5%
Answer Questions	30%

Grading Scale



- A** = (81 - 100) → **Sepuh**
- B+** = (75 - 80) → **Very Good**
- B** = (70 - 74) → **Good**
- C+** = (60 - 69) → **Fairly Good**
- C** = (55 - 59) → **Fair**
- D** = (41 - 54) → **Poor**
- E** = (0 - 40) → **Bro really...**

